

Recording Metrics & Benchmarks

Providing **quantitative proof** that your infrastructure meets **SLOs** under real-world conditions

Delivering a comprehensive **benchmark pack** with charts, tables, and logs to validate your ML infrastructure



Why Benchmarks Matter

Benchmarks provide the **concrete evidence** that elevates your infrastructure from theoretical to production-ready:

Demonstrate Maturity

Shows reviewers your system is built on solid engineering principles

Highlight Capabilities

Quantifies your [scalability](#), [efficiency](#), and [resilience](#)

Enable Optimization

Establishes baselines for future improvements



Capstone Requirement

"Numbers, not promises" is the expectation from reviewers

Core SLOs to Measure

1

Latency

p50/p95/p99 response times

Shows consistency across different load conditions

2

Throughput

Sustained QPS/RPS per GPU/cluster

Demonstrates maximum capacity under load

3

Error Rate

4xx/5xx % and timeout frequency

Indicates system reliability and robustness

1

Availability

Uptime % and failover success rate

Proves system resilience during disruptions

2

Cost Efficiency

\$/1k requests and GPU utilization %

Shows resource optimization and financial viability

3

Model Quality

Drift metrics and win-rate vs baseline

Ensures consistent ML performance

Test Setup

Environment & Workload

- **Testing Environment:** demo namespace (Day 358)
- **Workload Mix:** Realistic payload sizes and request distributions that mirror production patterns

Test Durations

- **Load Test:** 10-15 minutes
- **Stress/Spike Test:** 5 minutes
- **Soak Test:** 1-3 hours

Testing Toolkit



Load Generation

k6 / Locust for realistic traffic simulation



Metrics Collection

Prometheus + Grafana for visualization



GPU Benchmarking

Triton perf_analyzer for hardware throughput



Resilience Testing

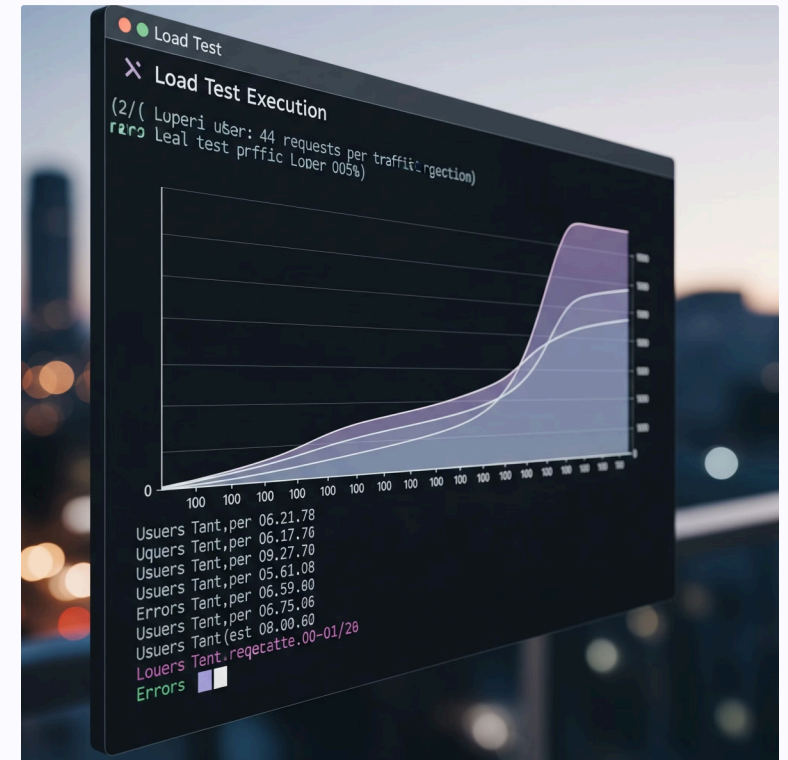
Chaos Mesh for failover simulations

Load Test Script Example

```
import http from 'k6/http';
export let options = {
  stages: [
    { duration: '2m', target: 100 },
    { duration: '5m', target: 200 },
    { duration: '2m', target: 0 },
  ],
  thresholds: {
    http_req_duration: ['p(95)<300'],
    http_req_failed: ['rate<0.001']
  }
};
export default function () {
  http.post(`${__ENV.BASE}/predict`,
    JSON.stringify({text: "benchmark"}),
    { headers: { 'Content-Type': 'application/json' } }
  );
}
```

Run command:

```
BASE=https://demo.capstone.local k6 run script.js --out json=results.json
```



This k6 script creates a gradual ramp-up to 200 concurrent users, sustains the load for 5 minutes, then ramps down - capturing the full response profile of your system.

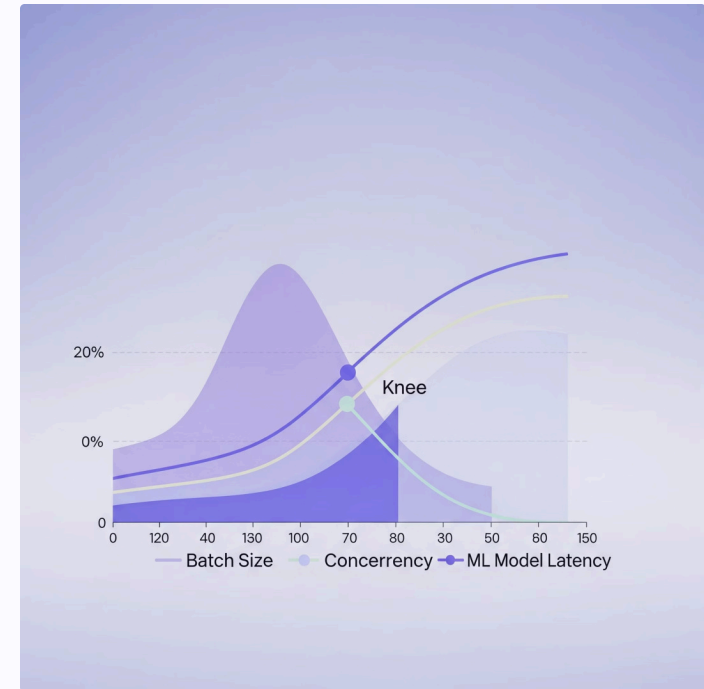
GPU Benchmarking with Triton

Triton Performance Analyzer

```
perf_analyzer -m sentiment_model \  
-b 32 -p 5000 \  
-u triton.demo.svc.cluster.local:8001
```

Benchmark Strategy

- Sweep across **batch sizes** (8, 16, 32, 64)
- Test varying **concurrency levels** (1, 5, 10, 20)
- Record complete **throughput/latency curve**
- Identify optimal operating point before latency degrades



Identify the "knee point" - the optimal throughput level before p95 latency spikes, indicating maximum sustainable capacity

Key Metrics to Capture



Latency Distribution

p50/p95/p99 showing service predictability



QPS vs Latency

Capacity knee curve showing maximum sustainable load



Error % vs Load

System stability under increasing pressure



GPU Utilization

Resource efficiency across throughput levels



\$/1k Requests

Financial metrics from Kubecost or billing export



Drift PSI Trend

Model stability over the demo window

These metrics together provide a **comprehensive view** of system performance, reliability, and efficiency

Example Benchmark Results

Test	RPS	p50 (ms)	p95 (ms)	p99 (ms)	Errors	GPU Util	Notes
Baseline	40	55	80	0%	20%	Golden	
Load	200	130	270	410	0.02%	65%	Meets SLO
Stress	600	220	730	1200	1.8%	95%	Past knee
Spike	400	160	350	590	0.2%	80%	Survives
Soak (3h)	150	135	295	510	0.05%	72%	No leaks

Interpreting Results

- **Load Test (200 RPS):** System comfortably handles expected traffic
- **Stress Test (600 RPS):** Identifies upper limits where errors increase
- **Spike Test (400 RPS):** Validates ability to handle sudden traffic surges
- **Soak Test (150 RPS for 3h):** Confirms stability over extended periods



Cost Benchmarking

Key Cost Metrics

GPU Utilization

Target >60% to ensure efficient resource usage

\$/Training Run

Compare spot vs. on-demand instances for cost optimization

\$/1k Inferences

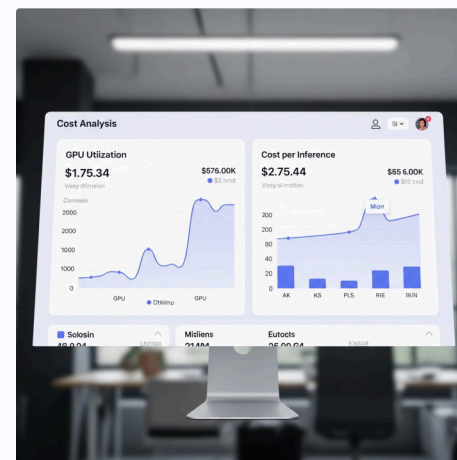
Measured under production-like load conditions

Spend Drift

Track actual costs against projected budget

Kubecost PromQL Example

```
sum(rate(container_gpu_usage_seconds_total[5m]))  
by (namespace)  
* on(namespace)  
group_left cost_per_gpu_hour
```



Availability Benchmark

Chaos Engineering Test

- **Scenario:** Kill 30% of service pods for 5 minutes
- **Purpose:** Validate automatic failover mechanisms

Success Criteria

- Traffic successfully reroutes to healthy replicas
- p95 latency increase < 25% from baseline
- Error rate remains below 0.1%
- **RTO:** Service stabilizes within SLA window



Recovery Time Objective (RTO)

Measure how quickly your system returns to stable performance after a disruption - a critical metric for highly available systems

Drift Benchmark

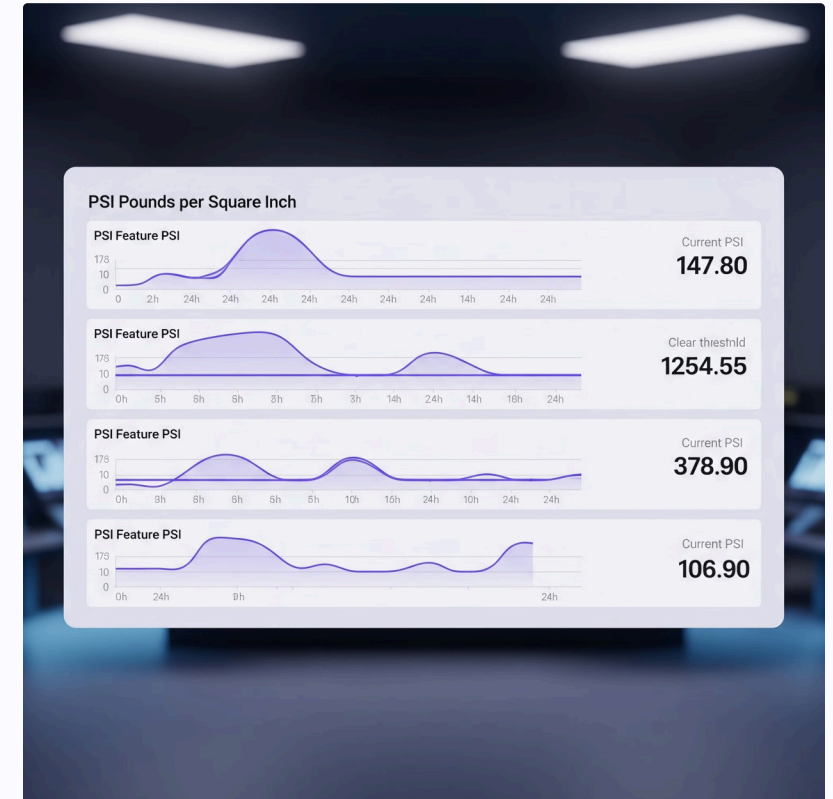
Drift Monitoring Strategy

- Run automated drift checks nightly using Evidently
- Record PSI (Population Stability Index) for top features
- Track Kolmogorov-Smirnov statistics for distribution changes
- Plot trend of drift score compared to established baseline

Acceptable Thresholds

PSI < 0.25 indicates feature distribution stability

- PSI < 0.1: Minimal drift
- PSI 0.1-0.25: Moderate drift (investigate)
- PSI > 0.25: Significant drift (requires action)

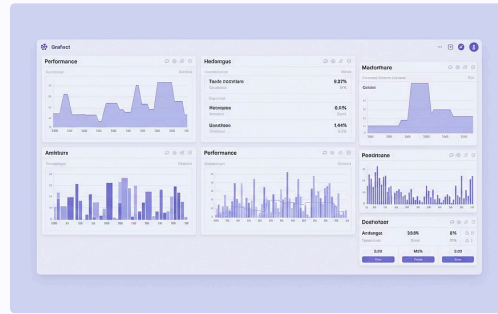


Recording Benchmark Artifacts



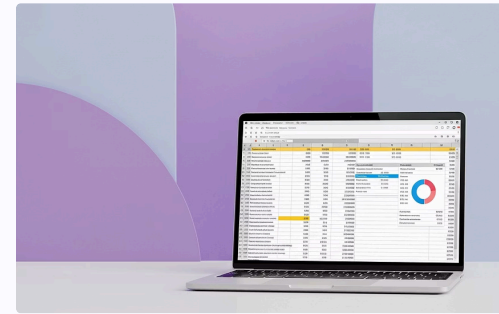
Raw Data

JSON exports from k6 load tests capturing complete performance data



Visualization

Grafana snapshots of API latency, GPU utilization, and drift panels



Tabular Data

CSV/XLS tables summarizing key benchmark metrics for easy review



Reports

Evidently drift reports and PDF summary charts for Capstone submission

Store all artifacts in a centralized location for easy access during Capstone defense

Dashboard Visualization

"Benchmarks prove infrastructure meets SLOs with significant headroom for growth"

Performance Metrics

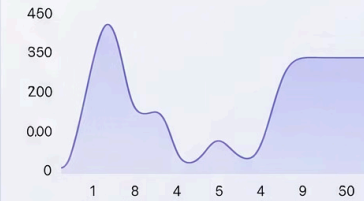
- QPS vs. Latency shows capacity headroom
- GPU Utilization demonstrates resource efficiency

Reliability Metrics

- Error % trend confirms system stability
- Drift PSI timeline validates model consistency

ML Infrastructure

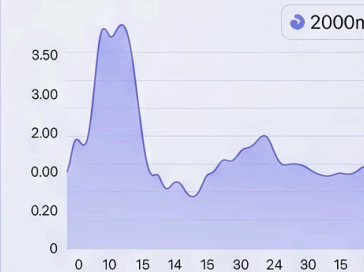
QPS Latency



GPU Utilization vs Throughput



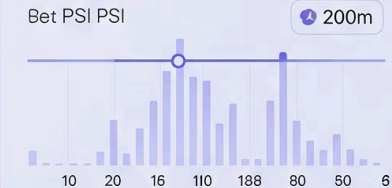
Error Percentage



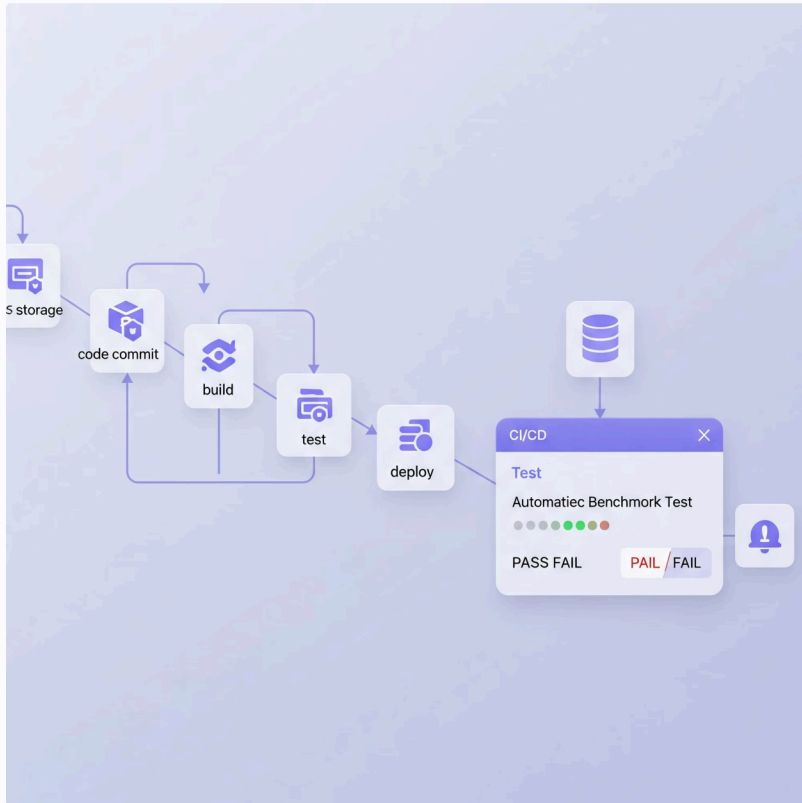
Underscore



Drift PSI



Automating Benchmarks in CI/CD



Nightly Benchmark Workflow

1. Run Automated Tests

- k6 baseline load test
- Drift check comparison

2. Store Results

- Upload artifacts to S3 bucket
- Archive in repository /benchmarks folder

3. Alert on Violations

- Latency p95 > 300ms
- Drift PSI > 0.25
- \$/1k requests exceeds threshold

Automation ensures your benchmark data remains current and violations are caught early



Capstone Day Checklist

Performance Metrics

- Latency p95/p99 at target load ✓
- Maximum QPS (knee point) documented ✓
- Error rate within SLA limits ✓

ML Performance

- Drift report included ✓
- Model quality metrics documented ✓

Reliability

- Availability drill logs attached ✓
- RTO measurements documented ✓

Resource Efficiency

- GPU utilization $\geq 60\%$ ✓
- Cost benchmarks recorded ✓

Summary

Benchmarks = Evidence

Your benchmarks demonstrate:

- **Reliability:** System meets latency and error rate SLOs
- **Resilience:** Infrastructure handles failures gracefully
- **Efficiency:** Resources are utilized optimally
- **Quality:** ML models maintain performance stability

Best Practices:

- Record comprehensive metrics across all SLOs
- Present results in easy-to-understand dashboards and tables
- Automate nightly tests to maintain current benchmarks



Discussion

Which benchmark will be hardest to meet in your Capstone?



Latency

Achieving consistent p95/p99 response times under variable load conditions



Drift Stability

Maintaining model performance as data distributions evolve over time



Cost Per Inference

Optimizing infrastructure spend while meeting performance requirements